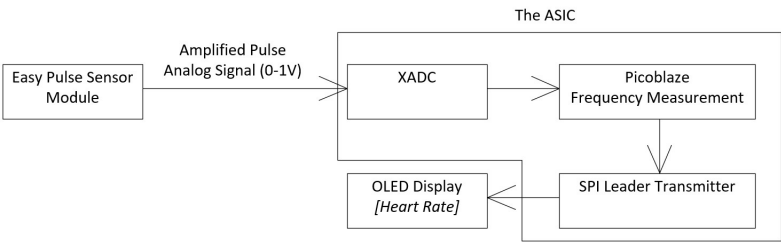


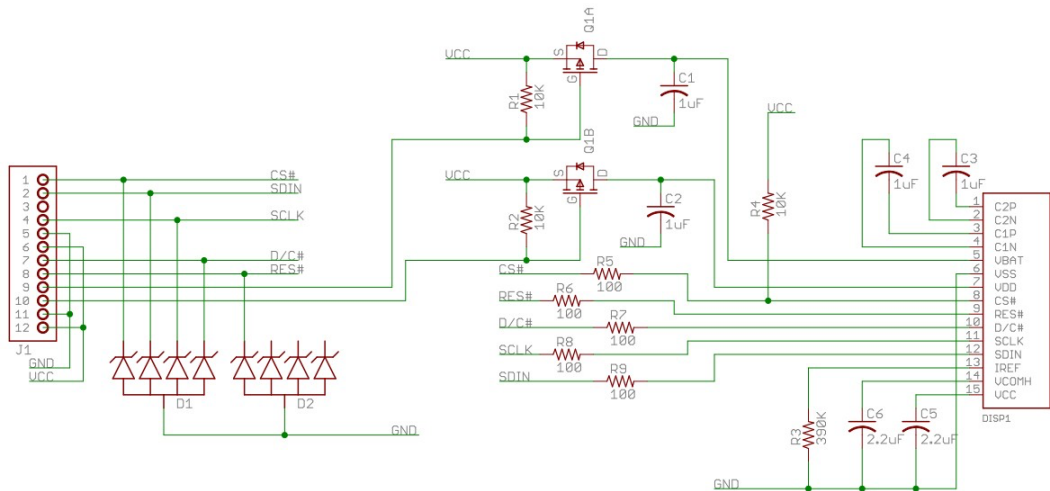
Spring 2026 CE433 Course Project: An Pulse Sensor ASIC Design

The goal of this projet is to design a pulse sensor ASIC that includes an off-chip amplifier/filter circuit, an on-chip ADC, Picoblaze CPU, and a SPI modul communicates with an OLED display module (Pmod OLED: 128 x 32 Pixel Monochromatic OLED Display).

The product diagram can be found below.



The Pmod OLED schematic can be found below.



The pinout table can be found below. The CS, MOSI, and SCK pins are standard SPI pins. The fourth pin is D/C which is for Data/Command Control.

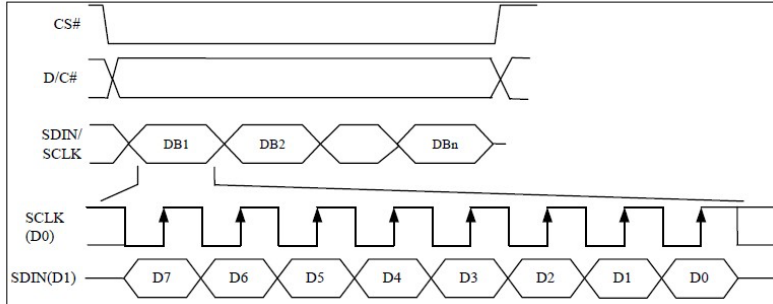
Pinout Description Table

Header J1					
Pin	Signal	Description	Pin	Signal	Description
1	CS	Chip Select	7	D/C	Data/Command Control
2	MOSI	Master-Out-Slave-In	8	RES	Power Reset
3	NC	Not Connected	9	VBATC	Vbat Battery Voltage Control
4	SCK	Serial Clock	10	VDDC	Vdd Logic Voltage Control
5	GND	Power Supply Ground	11	GND	Power Supply Ground
6	VCC	Power Supply (3.3V)	12	VCC	Power Supply (3.3V)

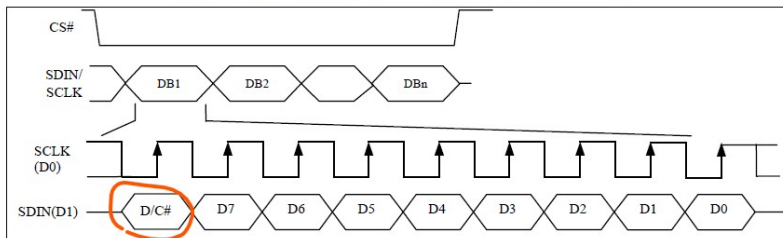
The D/C pin determines if the bytes are for the display or for the commands. The [SSD1306 datasheet](#) shows the details. From the datasheet, Section 8.1.3 MCU Serial Interface (4-wire SPI) describes how to use the fourth pin D/C. SDIN is shifted into an 8-bit shift register on every rising edge of SCLK in the order of D7, D6, ... D0. D/C# is sampled on every eighth clock and the data shift register is written to the Graphic Display Data RAM (GDDRAM) or command register in the same clock.

Table 8-4 : Control pins of 4-wire Serial interface

Function	E(RD#)	R/W#(WR#)	CS#	D/C#	D0
<u>Write command</u>	Tie LOW	Tie LOW	L	<u>L</u>	↑
<u>Write data</u>	Tie LOW	Tie LOW	L	<u>H</u>	↑

Figure 8-5 : Write procedure in 4-wire Serial interface mode

However, for the Section 8.1.4 MCU Serial Interface (3-wire SPI), it requires the D/C command to be sent along with data as D8. This requires a different driver in hardware and software to handle this bit.

Figure 8-6 : Write procedure in 3-wire Serial interface mode

The Pmod OLED example page provided an [example package](#) for the Nexys 3 board. We are using the Basys 3 board so some pins, connections, and the code need changes to be compatible with the board.

Open the top file of the design, the port declaration tells that DC is a fourth pin so it must use the 4-wire SPI timing diagram.

```

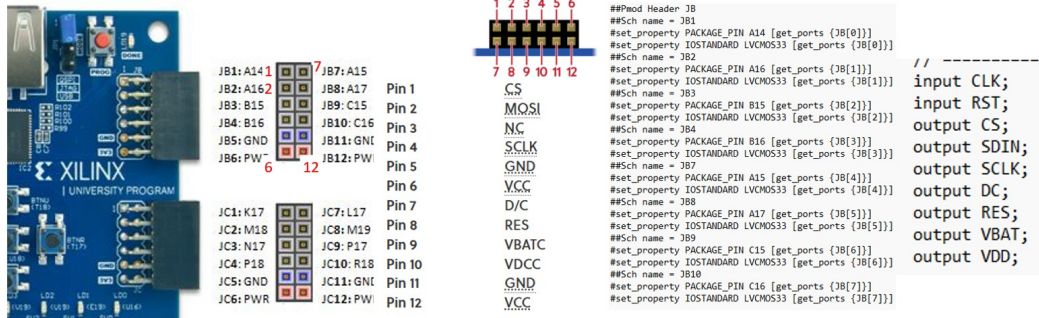
////////////////////////////////////
module PmodOLEDctrl(
    CLK,
    RST,
    CS,
    SDIN,
    SCLK,
    DC,
    RES,
    VBAT,
    VDD
);

// ===== Port
//
Declarations
// =====
input CLK;
input RST;
output CS;
output SDIN;
output SCLK;
output DC;
output RES;
output VBAT;
output VDD;

```

Bring the Picoblaze core to this design and write an assembly code to measure the digitized DO from the pulse sensor and count an average heart rate on the OLED display module.

The pins of the PmodOLED is mapped to the JB pins as follows:

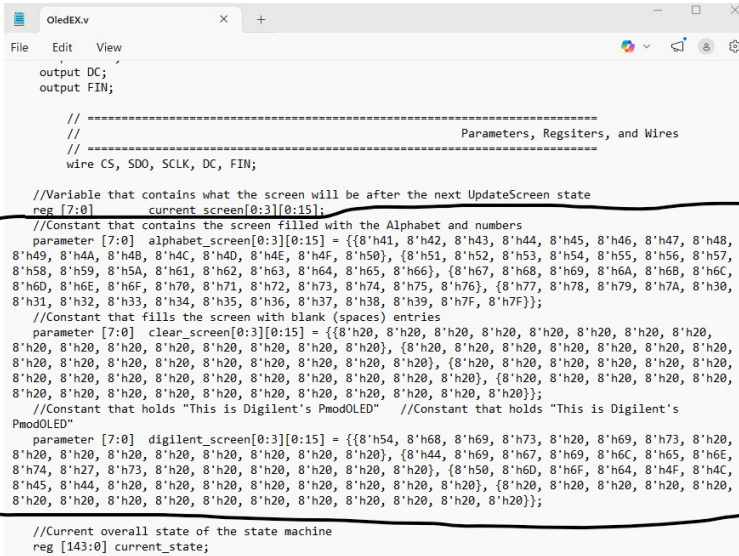


```

5
6 # Clock signal
7 set_property PACKAGE_PIN W5 [get_ports CLK]
8 set_property IOSTANDARD LVCMOS33 [get_ports CLK]
9 create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports CLK]
10
11 ## Switches
12 set_property PACKAGE_PIN V17 [get_ports RST]
13 set_property IOSTANDARD LVCMOS33 [get_ports RST]
14 set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
16 set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132 #Pmod Header JB
133 #Sch name = JB1
134 set_property PACKAGE_PIN A14 [get_ports {CS}]
135 set_property IOSTANDARD LVCMOS33 [get_ports {CS}]
136 #Sch name = JB2
137 set_property PACKAGE_PIN A16 [get_ports {SDIN}]
138 set_property IOSTANDARD LVCMOS33 [get_ports {SDIN}]
139 #Sch name = JB3
140 set_property PACKAGE_PIN B15 [get_ports {JB[2]}]
141 set_property IOSTANDARD LVCMOS33 [get_ports {JB[2]}]
142 #Sch name = JB4
143 set_property PACKAGE_PIN B16 [get_ports {SCLK}]
144 set_property IOSTANDARD LVCMOS33 [get_ports {SCLK}]
145 #Sch name = JB7
146 set_property PACKAGE_PIN A15 [get_ports {DC}]
147 set_property IOSTANDARD LVCMOS33 [get_ports {DC}]
148 #Sch name = JB8
149 set_property PACKAGE_PIN A17 [get_ports {RES}]
150 set_property IOSTANDARD LVCMOS33 [get_ports {RES}]
151 #Sch name = JB9
152 set_property PACKAGE_PIN C15 [get_ports {VBAT}]
153 set_property IOSTANDARD LVCMOS33 [get_ports {VBAT}]
154 #Sch name = JB10
155 set_property PACKAGE_PIN C16 [get_ports {VDD}]
156 set_property IOSTANDARD LVCMOS33 [get_ports {VDD}]
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177

```

The following content in OledEX.v must be replaced by the initial begin/end block listed below since the format is not supported by Vivado.



```

output DC;
output FIN;

// =====
//                                     Parameters, Registers, and Wires
// =====
wire CS, SDO, SCLK, DC, FIN;

//Variable that contains what the screen will be after the next UpdateScreen state
reg [7:0] current_screen[0:3][0:15];

//Constant that contains the screen filled with the Alphabet and numbers
parameter [7:0] alphabet_screen[0:3][0:15] = {{8'h41, 8'h42, 8'h43, 8'h44, 8'h45, 8'h46, 8'h47, 8'h48,
8'h49, 8'h4A, 8'h4B, 8'h4C, 8'h4D, 8'h4E, 8'h4F, 8'h50}, {8'h51, 8'h52, 8'h53, 8'h54, 8'h55, 8'h56, 8'h57,
8'h58, 8'h59, 8'h5A, 8'h5B, 8'h5C, 8'h5D, 8'h5E, 8'h5F, 8'h60}, {8'h61, 8'h62, 8'h63, 8'h64, 8'h65, 8'h66, 8'h67,
8'h68, 8'h69, 8'h6A, 8'h6B, 8'h6C, 8'h6D, 8'h6E, 8'h6F, 8'h70}, {8'h71, 8'h72, 8'h73, 8'h74, 8'h75, 8'h76, 8'h77,
8'h78, 8'h79, 8'h7A, 8'h7B, 8'h7C, 8'h7D, 8'h7E, 8'h7F}};

//Constant that fills the screen with blank (spaces) entries
parameter [7:0] clear_screen[0:3][0:15] = {{8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20,
8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}, {8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20,
8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}, {8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20,
8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}, {8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20,
8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}};

//Constant that holds "This is Digilent's PmodOLED" //Constant that holds "This is Digilent's
PmodOLED"
parameter [7:0] diligent_screen[0:3][0:15] = {{8'h54, 8'h68, 8'h69, 8'h73, 8'h20, 8'h69, 8'h73, 8'h20,
8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}, {8'h44, 8'h69, 8'h67, 8'h69, 8'h6C, 8'h65, 8'h6E,
8'h74, 8'h27, 8'h73, 8'h20, 8'h20, 8'h20, 8'h20}, {8'h58, 8'h6D, 8'h6F, 8'h64, 8'h4F, 8'h4C,
8'h45, 8'h44, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}, {8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20,
8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20, 8'h20}};

//Current overall state of the state machine
reg [143:0] current_state;

```

initial begin

```

// ===== Alphabet Screen =====
// Row 0
alphabet_screen[0][0]=8'h41; alphabet_screen[0][1]=8'h42;
alphabet_screen[0][2]=8'h43; alphabet_screen[0][3]=8'h44;
alphabet_screen[0][4]=8'h45; alphabet_screen[0][5]=8'h46;
alphabet_screen[0][6]=8'h47; alphabet_screen[0][7]=8'h48;
alphabet_screen[0][8]=8'h49; alphabet_screen[0][9]=8'h4A;
alphabet_screen[0][10]=8'h4B; alphabet_screen[0][11]=8'h4C;
alphabet_screen[0][12]=8'h4D; alphabet_screen[0][13]=8'h4E;
alphabet_screen[0][14]=8'h4F; alphabet_screen[0][15]=8'h50;

// Row 1
alphabet_screen[1][0]=8'h51; alphabet_screen[1][1]=8'h52;
alphabet_screen[1][2]=8'h53; alphabet_screen[1][3]=8'h54;
alphabet_screen[1][4]=8'h55; alphabet_screen[1][5]=8'h56;
alphabet_screen[1][6]=8'h57; alphabet_screen[1][7]=8'h58;
alphabet_screen[1][8]=8'h59; alphabet_screen[1][9]=8'h5A;
alphabet_screen[1][10]=8'h5B; alphabet_screen[1][11]=8'h5C;
alphabet_screen[1][12]=8'h5D; alphabet_screen[1][13]=8'h5E;
alphabet_screen[1][14]=8'h5F; alphabet_screen[1][15]=8'h60;

// Row 2
alphabet_screen[2][0]=8'h61; alphabet_screen[2][1]=8'h62;
alphabet_screen[2][2]=8'h63; alphabet_screen[2][3]=8'h64;
alphabet_screen[2][4]=8'h65; alphabet_screen[2][5]=8'h66;
alphabet_screen[2][6]=8'h67; alphabet_screen[2][7]=8'h68;
alphabet_screen[2][8]=8'h69; alphabet_screen[2][9]=8'h6A;
alphabet_screen[2][10]=8'h6B; alphabet_screen[2][11]=8'h6C;
alphabet_screen[2][12]=8'h6D; alphabet_screen[2][13]=8'h6E;
alphabet_screen[2][14]=8'h6F; alphabet_screen[2][15]=8'h70;

// Row 3
alphabet_screen[3][0]=8'h71; alphabet_screen[3][1]=8'h72;
alphabet_screen[3][2]=8'h73; alphabet_screen[3][3]=8'h74;
alphabet_screen[3][4]=8'h75; alphabet_screen[3][5]=8'h76;
alphabet_screen[3][6]=8'h77; alphabet_screen[3][7]=8'h78;
alphabet_screen[3][8]=8'h79; alphabet_screen[3][9]=8'h7A;
alphabet_screen[3][10]=8'h7B; alphabet_screen[3][11]=8'h7C;
alphabet_screen[3][12]=8'h7D; alphabet_screen[3][13]=8'h7E;
alphabet_screen[3][14]=8'h7F; alphabet_screen[3][15]=8'h80;

```

```
alphabet_screen[3][14]=8'h7F; alphabet_screen[3][15]=8'h7F;
```

```
// ===== Clear Screen =====
```

```
for (r = 0; r < 4; r = r + 1)
  for (c = 0; c < 16; c = c + 1)
    clear_screen[r][c] = 8'h20;
```

```
// ===== Diligent Screen =====
```

```
// Row 0: "This is"
```

```
diligent_screen[0][0]=8'h54; diligent_screen[0][1]=8'h68;
diligent_screen[0][2]=8'h69; diligent_screen[0][3]=8'h73;
diligent_screen[0][4]=8'h20; diligent_screen[0][5]=8'h69;
diligent_screen[0][6]=8'h73; diligent_screen[0][7]=8'h20;
diligent_screen[0][8]=8'h20; diligent_screen[0][9]=8'h20;
diligent_screen[0][10]=8'h20; diligent_screen[0][11]=8'h20;
diligent_screen[0][12]=8'h20; diligent_screen[0][13]=8'h20;
diligent_screen[0][14]=8'h20; diligent_screen[0][15]=8'h20;
```

```
// Row 1: "Diligent's"
```

```
diligent_screen[1][0]=8'h44; diligent_screen[1][1]=8'h69;
diligent_screen[1][2]=8'h67; diligent_screen[1][3]=8'h69;
diligent_screen[1][4]=8'h6C; diligent_screen[1][5]=8'h65;
diligent_screen[1][6]=8'h6E; diligent_screen[1][7]=8'h74;
diligent_screen[1][8]=8'h27; diligent_screen[1][9]=8'h73;
diligent_screen[1][10]=8'h20; diligent_screen[1][11]=8'h20;
diligent_screen[1][12]=8'h20; diligent_screen[1][13]=8'h20;
diligent_screen[1][14]=8'h20; diligent_screen[1][15]=8'h20;
```

```
// Row 2: "PmodOLED"
```

```
diligent_screen[2][0]=8'h50; diligent_screen[2][1]=8'h6D;
diligent_screen[2][2]=8'h6F; diligent_screen[2][3]=8'h64;
diligent_screen[2][4]=8'h4F; diligent_screen[2][5]=8'h4C;
diligent_screen[2][6]=8'h45; diligent_screen[2][7]=8'h44;
diligent_screen[2][8]=8'h20; diligent_screen[2][9]=8'h20;
diligent_screen[2][10]=8'h20; diligent_screen[2][11]=8'h20;
diligent_screen[2][12]=8'h20; diligent_screen[2][13]=8'h20;
diligent_screen[2][14]=8'h20; diligent_screen[2][15]=8'h20;
```

```
// Row 3: blank
```

```
for (c = 0; c < 16; c = c + 1)
  diligent_screen[3][c] = 8'h20;
```

```
end
```

In addition to the modifications above, you also need to create a memory 'charLib' to store the charLib.coe data. You can tell that charLib is instantiated in the module

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☒ Show disabled ports

Component Name: charLib

Basic Port A Options Other Options Summary

Interface Type: Native ☐ Generate address interface with 32 bits

Memory Type: Single Port ROM ☐ Common Clock

ECC Options

ECC Type: No ECC

☐ Error Injection Pins: Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits): 9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information.

Algorithm: Minimum Area

Primitive: 8kx2

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☒ Show disabled ports

Component Name: charLib

Basic **Port A Options** Other Options Summary

Memory Size

Port A Width: 8 Range: 1 to 4608 (bits)

Port A Depth: 61440 Range: 2 to 1048576

The Width and Depth values are used for Read Operation in Port A

Operating Mode: Write First Enable Port Type: Always Enabled

Port A Optional Output Registers

☒ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

Port A Output Reset Options

☐ RSTA Pin (set/reset pin) Output Reset Value (Hex): 0

☐ Reset Memory Latch Reset Priority: CE (Latch or Register Enable)

READ Address Change A

☐ Read Address Change A

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☒ Show disabled ports

Component Name: charLib

Basic Port A Options **Other Options** Summary

Pipeline Stages within Mux: 0 Mux Size: 15x1

Memory Initialization

☒ Load Init File

Coe File: i26/pmodoledctrl_verlogiPmodOLED_Source/charLib.coe

☐ Fill Remaining Memory Locations

Remaining Memory Locations (Hex): 0

Structural/UniSim Simulation Model Options

Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.

Collision Warnings: All

Behavioral Simulation Model Options

☐ Disable Collision Warnings ☐ Disable Out of Range Warnings

Here is the result of the example from Digilent.

Documentation

- EMC Disclaimer, Digilent Development and Evaluation Kits

Example Projects

Microprocessor

- Installing the Digilent Core for Arduino
- Library and MPLAB Example
- Library and MPLAB Example
- Behind Tool Anti-Creep Alert System - Community Project
- Wearable Wellness System - Community Project
- Health and Security Cloud System - Community Project
- 3D road mapping automobile - Community Project
- Using the Pmod OLED with Arduino Uno - Application note

Programmable Logic

- News 3 Version Example - ISE H.2**
- News 3 VHDL Example - ISE H.2
- News 4 VHDL Example - Vivado H.1
- Sounding Rocket Avionics with FPGA - Community Project
- Getting Started with Digilent Pmod IPs

Primary IC
Reference
Manual
Schematic

SSD1306
Pmod OLED Reference
Manual

J1 Pinout

Pin 1 CS
Pin 2 MOSI
Pin 3 NC
Pin 4 SCLK
Pin 5 GND
Pin 6 VCC
Pin 7 D/C
Pin 8 RES
Pin 9 VBAIC
Pin 10 VDDC
Pin 11 GND

Yiyan Li



Watch on

Tasks:

- Task 1: Turn the PmodOLEDCtrl.v and the submodules into a Basys3 compatible design and display any characters on the OLED to prove its functionality. Picoblaze involved, only use the code from the example and make it Basys3 compatible. (50 points)
- Task 2: Add Picoblaze to the design, probe AO, read the XADC's output and display the binary form on the on-board LEDs. (50 points)
- Task 3: Use assembly and Picoblaze for frequency measurement and display heart rate on the OLED and at the same time, display realtime waveforms on plot. (50 points)

References:

- [Pmod OLED product page](#)
- [Pmod OLED support page](#)
- [Pmod OLED examples page](#)
- [SSD1306 datasheet](#)